posterior distribution produced by models in pilot simulations when early stopping was not required. Forced early stopping is detrimental to performance; we conjecture that forced stopping strictly harms performance because the latter steps in the pause sequence contribute less to the loss, and thus they may be inadequately trained to be used effectively. The downside of eliminating the time pressure is that models may waste pause tokens unnecessarily. However, we show shortly that this concern is not observed.

The comparison of CYB-AP models also indicates that incorporating even weak discounting ($\gamma = 0.99$) is harmful. Discounting imposes a pressure on the model to use additional steps only when doing so improves performance according to the speed/accuracy trade off represented by γ—one percent in accuracy per step if $\gamma = 0.99$. We found in pilot experiments that greater discounting led to strictly worse model performance.

CYB-VA performs poorly for three different target stop-time distributions, ρ. We considered incorporating a scaling coefficient on the KL term, but CYB-VA's elegance stems from the fact that such a coefficient should not be required.

For CYB-DP, which penalizes deviation from a target stop-time distribution, we tested two penalty coefficients, $\alpha \in \{0.01, 0.10\}$ and three target distributions ρ, with ratios 1:1:1:1, 4:1:1:4, and 11:3:3:3. The smaller coefficient yielded better performance, and the similar performance over ρ suggests that the penalty had little impact. If the penalty is not shaping learning, CYB-DP becomes equivalent to CYB-AP with no world-induced early stopping. Thus, we obtain clear evidence that CYB with no explicit time cost and no forced early termination yields optimal performance. With no time pressure, CYB-AP might simply use the maximum number of computation steps for every token. We show later that it does not take such a lazy approach.

Data efficiency of CYB-AP. While CYB-AP reduces perplexity, developing intuitions about the significance of this reduction is challenging. We aim to provide such intuition by casting the improvement in terms of data efficiency. Figure 6 presents evaluation perplexity of various models fine tuned in different data regimes. The regimes labeled *1×1* and *2×1* correspond to one and two passes through our 6.4M training sequences, respectively. The regimes labeled *1×2* and *1×3* correspond to data sets that are two and three times as large, respectively. CYB-AP requires just 33% of the data to match performance of the baseline model, and less than 66% of the data to beat performance of TBYS with three pauses. Interestingly, the performance gap between CYB-AP and the alternative methods remains intact as data sets grow, indicating CYB-AP's gains are a genuine improvement in learning efficiency, not an artifact of data saturation.

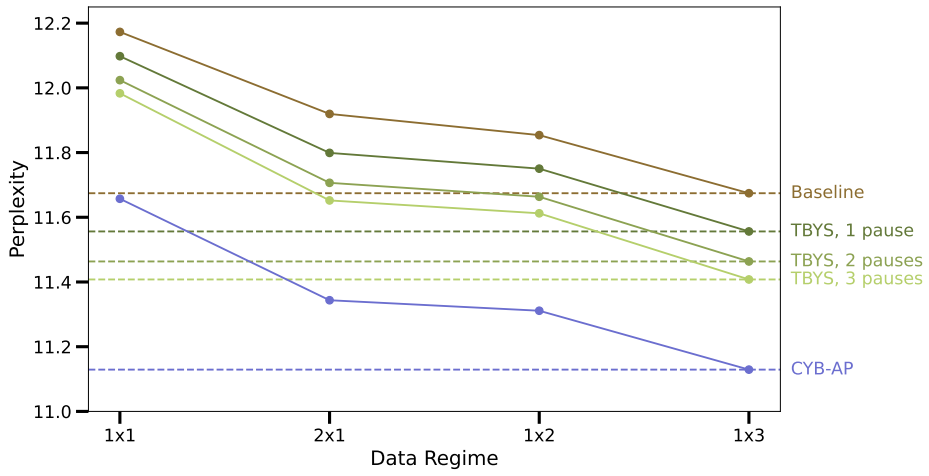

Figure 6: C4 evaluation set perplexity for various fine-tuned models based on a given data regime. *1x1* and *2x1* refer to 1 and 2 passes through the fine tuning set, respectively. *1x2* and *1x3* refer to a single pass through 2 or 3 times the dataset size of 1x1. The dashed lines indicate the perplexity of each model in the 1x3 training regime.

Pause utilization. We now turn to analyzing how CYB learns to utilize pauses. For each token in the evaluation set, we can use a model's <DON'T KNOW> output to compute the *latency* distribution (Equation 2), i.e., the distribution over which step the model reads out its response. Figure 7 presents statistics of the latency distribution for four different variants of CYB-AP. The blue curves show a probability density over the expected latency for individual tokens. The purple curves show the latency distribution averaged over tokens.

When the model has no external pressure to prepare for an early stop—the two leftmost columns of Figure 7 where $\omega = 0:0:0:1$—it nonetheless does not request the maximum number of pauses. This finding is surprising given that additional pause steps should not *harm* performance. When slower responses are penalized via the accuracy discounting factor γ (second column in Figure 7) the model shifts the mean latency distribution lower, as one would expect: with discounting, delaying a response is penalized unless the delay results in a corresponding boost in accuracy.

The rightmost two columns of Figure 7 are variants of the model with early forced stops which allow the model to use the maximum latency on only 25% and 40% of tokens, respectively. With this form of time pressure, the model actually shifts its latency distribution to the *right* relative to the distribution in column 1, which better matches the stop time distribution ω. Given that the unconstrained model, with $\gamma = 1.0$ and $\omega = 0:0:0:1$, achieves the best performance with a low latency, we choose this variant moving forward.

Analogous distribution figures for CYB-DP and CYB-VA are presented in Figures 8 and 9. CYB-VA adheres to the prior distribution and in general shows little variance indicating that it is are insensitive to the specific token and context, which is a bad sign for an adaptive-pause method. CYB-DP for

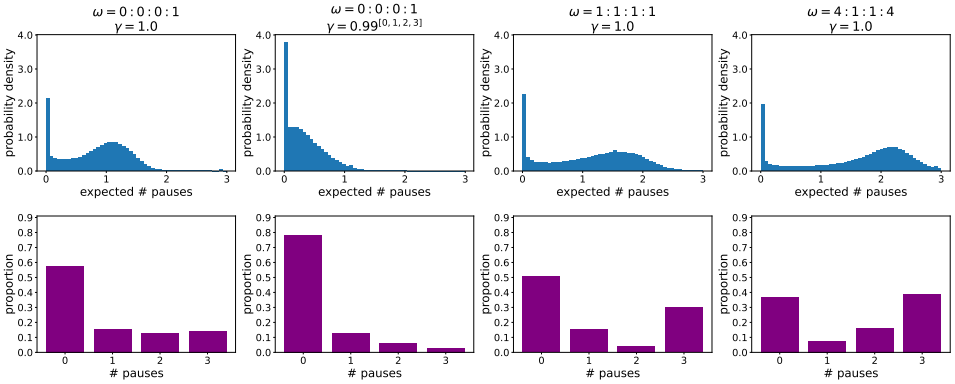

Figure 7: **Latency distribution for four parameterizations of CYB-AP**. Blue histograms show the distribution over evaluation tokens of the expected latency (the number of pause steps utilized). Purple bar graphs show the latency distribution averaged across tokens.

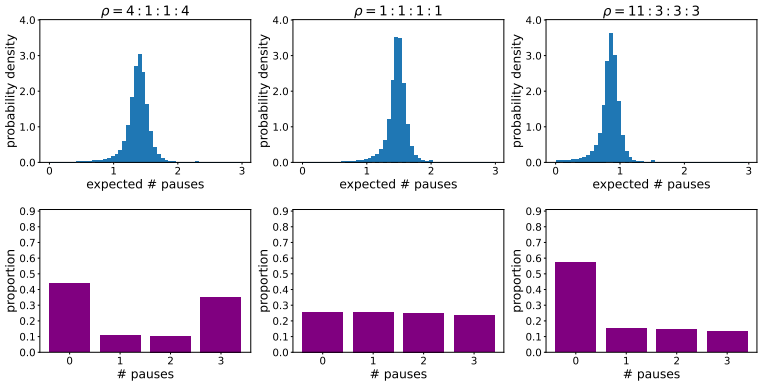

Figure 8: Distribution of stopping times for CYB-VA. Blue histograms show the distribution over evaluation tokens of the expected number of pause steps. Purple bar graphs show the distribution over the number of pause steps across tokens.

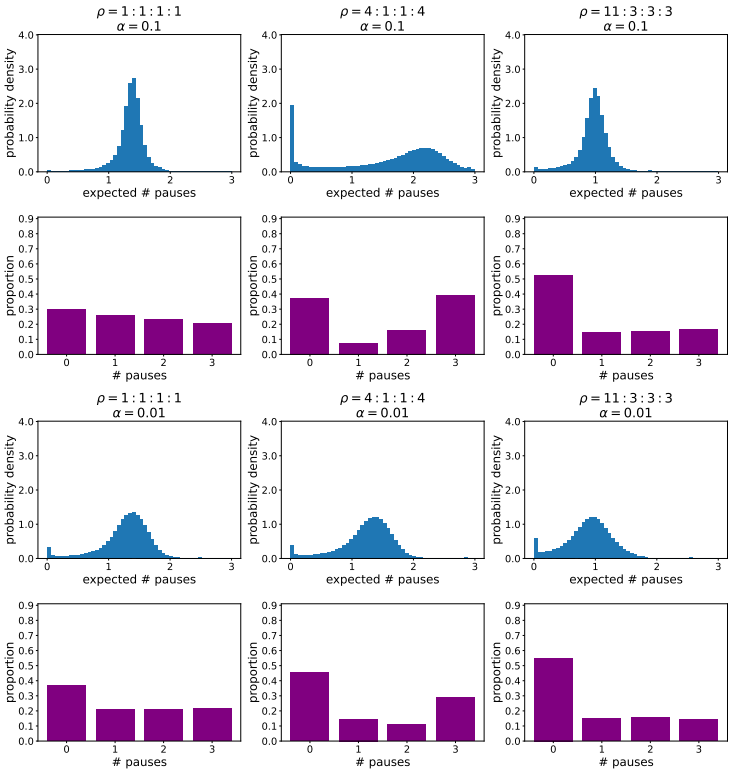

Figure 9: Distribution of stopping times for CYB-DP with $\alpha = 0.1$ and $\alpha = 0.01$. Blue histograms show the distribution over evaluation tokens of the expected number of pause steps. Purple bar graphs show the distribution over the number of pause steps across tokens.

large $\alpha = 0.1$ behaves closer to CYB-VA and for smaller $\alpha = 0.01$ prefers longer latencies than CYB-AP. However, longer latencies do not result in a lower perplexity compared to CYB-AP.

Calibration of model confidence and accuracy. If successful, CYB should train models to self-calibrate: <DON'T KNOW> should have high probability when a delay in responding will improve model output. To be successful, the model must implicitly predict its own accuracy at subsequent steps. That is, the probability of responding <DON'T KNOW> at step i, d_i, should be large when the target probability is higher at the next step, i.e., $t_{i+1} > t_i$. Figure 10 shows joint density maps of <DON'T KNOW> probability (along ordinate) and the improvement in accuracy obtained by a one-step delay in responding (along abscissa). The three density maps correspond to steps $i \in \{0, 1, 2\}$. A positive correlation indicates that the model can predict when it will benefit by the delay. Over the evaluation set, the Spearman correlation coefficient between d_i and $t_{i+1} - t_i$ is 0.383, 0.157, and 0.271 for steps 0-2, respectively, all reliably nonzero ($p < 0.001$). The correlations are only slightly stronger with the discounting variant of CYB-AP ($\gamma = 0.99^{\{0,1,2,3\}}$)—0.381, 0.208, and 0.309—indicating that penalizing delayed responses does not appear to make the model much better at determining when to pause.

Pause examples. Figures 11-13 present sample evaluation token sequences. Each token's background is colored to indicate the expected number of pause steps requested by the model in order to predict that token. (That is, the coloring reflects the processing time for the preceding token.) The coloring ranges from white to dark blue, corresponding to zero pause steps (1 total step) up to 3 pause steps (4 total steps).

The most obvious feature of the pause pattern is that it is not uniform but varies on a token-by-token basis. For example, the pause following the token 'silent' in Figure 11 tends to be long; the pause following the apostrophe token (') tends to be short. The pause duration following a particular input